COMPANION SLIDES · TUTORIAL/MBA_FROM_SCRATCH.IPYNB

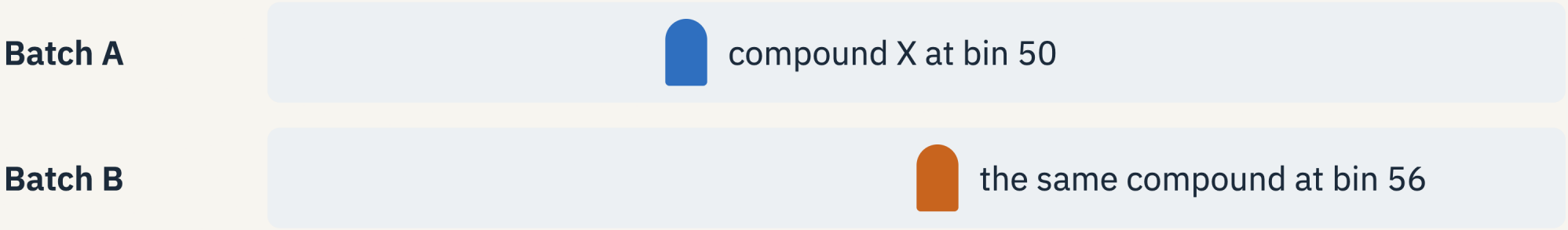
The joint model, step by step

How we line up retention times across many GC–MS batches, explained from the linear algebra up, with a refresher wherever one helps.

Every example uses the same small numbers as the notebook, so you can check each one by hand.

1 · THE PROBLEM

The same compound comes out at a different time in each batch



Compare bin 50 with bin 50 and you compare the compound in A with **background** in B. Every later statistic is then wrong.

We need a function that says **“bin 56 in B is bin 50 in A”**, at every time in the run. That function is a **warp**.

Notebook § 1 · One bin is 0.225 min, so this 6-bin shift is 1.35 min. The shift changes along the run: that is drift.

THE BIG IDEA

Give every batch one warp into a shared clock



The map from batch k to batch l is **$g(k \to l) = f_l^{-1}(f_k(t))$**: go into the shared clock, then undo batch l's warp.

Every route passes through the same clock, so **two routes between the same batches can never disagree.** Fitting each pair separately has no such guarantee.

Notebook § 2 and § 12 · f_l^{-1} is the inverse: the function that undoes f_l.

REFRESHER

Vectors, matrices and the one multiplication we need

Vector

A list of numbers.

`v = (0, 2, 1, 3)`

Matrix

A table of numbers: rows and columns. We will stack one row per equation.

Matrix × vector

Take each row, multiply it by the vector entry by entry, and add up. One number per row.

Worked example: one row times v

$$(0, \tfrac{1}{2}, \tfrac{1}{2}, 0) \cdot (0, 2, 1, 3) = 0\cdot 0 + \tfrac{1}{2}\cdot 2 + \tfrac{1}{2}\cdot 1 + 0\cdot 3 = 1.5$$

Notebook § 3 · In NumPy, matrix × vector is written B @ v.

2 · A WARP IS JUST A FUNCTION

A warp is the identity plus a small drift

$$f(t) = t + d(t)$$

The drift d is how far the batch's clock is from the shared clock. No drift means $d = 0$, and small drifts are small numbers.

Allowed: d(t) = 2 + 0.1t

f(10) = 10 + 3 = 13, f(20) = 20 + 4 = 24. The slope is 1.1, so later stays later.

Not allowed: d(t) = −1.5t

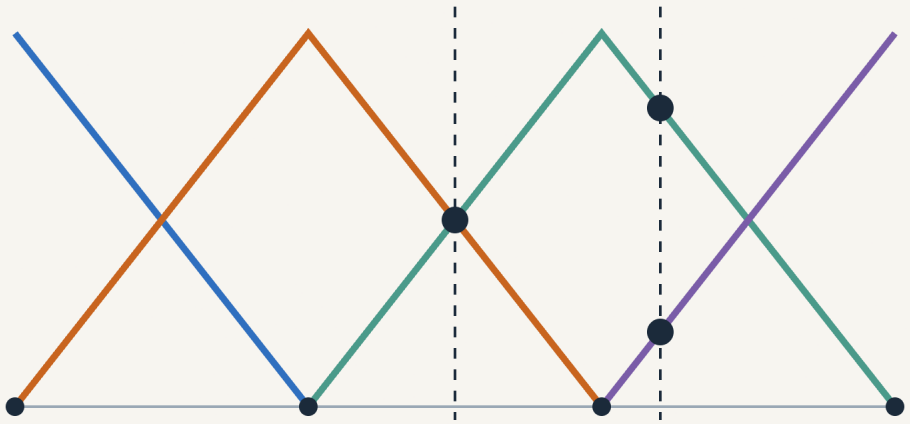
The slope is 1 − 1.5 = −0.5. Time runs backwards, so two compounds would swap order.

The rule we will enforce: the **slope stays between 0.5 and 2**. No batch runs more than twice as fast, or twice as slow, as the shared clock.

Notebook § 2 · Slope = how much f rises per bin. Positive everywhere = "monotone": order is preserved.

3 · KNOTS AND THE HAT BASIS

Describe the drift by its values at a few knots



Each coloured “hat” is one knot’s weight (height 1 at its own knot). Knots at t = 0, 10, 20, 30; dashed lines at t = 15 and t = 22.

Drift at the knots: **d = (0, 2, 1, 3)**. Between knots, join the dots with straight lines.

t = 15 is halfway between knots 1 and 2:

`B(15) = (0, ½, ½, 0) → d(15) = 1.5`

t = 22 is 2 bins from knot 2 and 8 from knot 3. The nearer knot gets more weight:

`B(22) = (0, 0, 0.8, 0.2) → d(22) = 1.4`

Interpolation is a row of weights times the knot values, the multiplication from the refresher. That is what keeps the whole fit linear.

Notebook § 3 · The real model uses 41 knots, about 1.1 min apart. Code: _hat_basis(t, knots).

4 · PARAMETERISING BY INCREMENTS

Store the first value and the steps, not every value

Keep **c**, the drift at the first knot, and the **steps δ** between neighbouring knots. Adding up the steps gives back every knot value.

$$\theta = (c, \delta_1, \delta_2, \delta_3) = (0, 2, -1, 2)$$
$$\text{running sums} \rightarrow d = (0, 2, 1, 3)$$

“Running sums” is a matrix too: **L**, with ones on and below the diagonal. So $d = L\theta$.

L for 4 knots

1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1

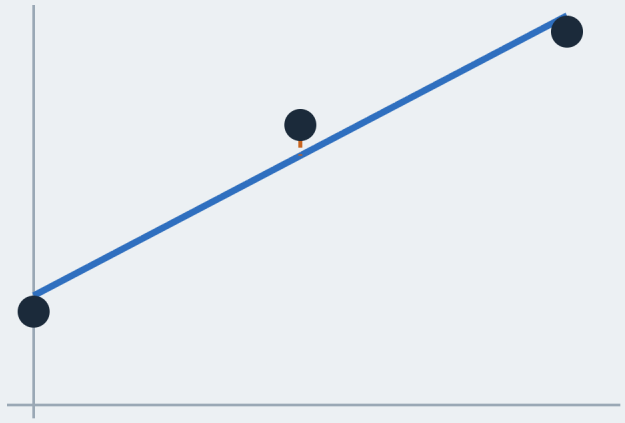
Row 3 · $\theta = 0 + 2 - 1 + 0 = 1$

Why bother? The slope between two knots is $1 + \delta / \Delta u$. With knots $\Delta u = 10$ apart, “slope between 0.5 and 2” becomes $-5 \le \delta \le 10$: a limit on one number at a time, which the solver handles easily.

Notebook § 4 · Code: Lcum = np.tril(np.ones((M, M))); bounds lb, ub. c itself has no limit.

REFRESHER · 5 · LEAST SQUARES

More equations than unknowns? Minimise the squared misses



Points (0, 1), (1, 3), (2, 4). No line passes through all three. Orange dashes: the misses we square and add up.

Model $y = m \cdot x + c$. One row per point, written $\mathbf{X\beta \approx y}$:

`X = [[0, 1], [1, 1], [2, 1]] β = (m, c) y = (1, 3, 4)`

The best β solves the **normal equations** $X^TX\beta = X^Ty$. (X^T, the *transpose*, swaps rows and columns.)

```
5m + 3c = 11
3m + 3c = 8 → m = 1.5, c ≈ 1.17
```

The joint model is this same calculation, with thousands of rows (anchors).

Notebook § 5 · Subtract the two equations: 2m = 3. In NumPy: np.linalg.lstsq(X, y).

REFRESHER · 5 · SINGULAR MATRICES

Sometimes there is no single best answer

Two columns say the same thing

`X = [[1, 1], [2, 2]]`

$X^{T}X$ = `[[5, 5], [5, 5]]`

The model is really y = (m + c)·x. The data pin down the **sum** m + c, never m and c apart.

The test: the determinant

For a 2 × 2 matrix [[a, b], [c, d]] it is **ad – bc**. Here 5·5 – 5·5 = 0.

Zero means the matrix can't be inverted ("singular"), so the normal equations have endless solutions.

The joint model hits exactly this problem: the data cannot tell where the shared clock *starts*. Slide "The gauge" shows the fix.

Notebook § 5 · A direction the data ignore is called the null space: Xv = 0.

6–7 · THE DESIGN MATRIX

Each anchor becomes one row of one big equation

An **anchor**: the same compound, matched by its spectrum, at time *a* in batch k and *b* in batch l. Both must land at the same shared time: $\mathbf{f_k(a) = f_l(b)}$.

Put every batch's unknowns in one long vector θ, in **tiles**: one tile of knot values per batch. The anchor only touches the tiles of its own two batches.

Example: 2 batches, knots at 0, 100, 200. An anchor at *a* = 50 in batch 0 and *b* = 150 in batch 1.

batch 0's tile: +B(50)L			batch 1's tile: −B(150)L			
1	0.5	0	-1	-1	-0.5	target y = b - a = 100

Stack one row per anchor into **X**. The misses are **r = Xθ − y**: the least-squares problem from the refresher.

Notebook § 6–7 · Each miss is $r = a - b + d_k(a) - d_l(b)$ (paper Eq. 3). The fish data give 25,011 rows.

8 · THE GAUGE

The data can’t tell where the shared clock starts

Anchor: a = 50 in batch 0, b = 56 in batch 1. The miss is $r = a - b + d_0 - d_1$.

```
d₀ = 0, d₁ = -6: 50 - 56 + 0 - (-6) = 0
add 1 to both: 50 - 56 + 1 - (-5) = 0 same miss
```

Anchors only say where batches sit **relative to each other**. Shift every drift by the same constant and nothing changes, so X^TX is singular, exactly as in the refresher. We add one extra rule:

Reference gauge (used in the paper)

Pin one batch’s drift to 0. The shared clock is then that batch’s own clock: sep2015, the batch with the most anchors.

Mean gauge

Make all the drifts add to 0 at every knot. The shared clock is the batches’ average.

Notebook § 8 · Only a constant shift is truly invisible. The two gauges give maps about 0.2 bins apart in the toy, so the paper leaves the choice open.

9 · THE SMOOTHNESS PENALTY

Straight lines are free; every bend costs

The **second difference** of three neighbours x_0, x_1, x_2 is $x_0 - 2x_1 + x_2$. It measures how much the line bends there.

```
(0, 2, 1, 3) → (-3, 3) bends
(0, 1, 2, 3) → (0, 0) straight
```

We add **λ × (sum of squared second differences)** to the misses. λ sets how much we care about bending.

Trick: a penalty is just **extra rows** with target 0, so it is still plain least squares.

λ too small

The drift wiggles to chase noise in the anchors.

λ about right

Follows the real drift, ignores the noise. Chosen from data (slide “Choosing λ”).

λ too large

Only a straight line is cheap enough, so real bumps are missed.

Notebook § 9 · λ is scaled by the number of anchors, so one λ means the same smoothness with 600 or 25,000 anchors. Paper Eq. 4.

10 · BOUNDS

Bounds keep every warp running forwards

Two mismatched anchors can tempt the fit to run a batch's clock backwards for a stretch. The increments trick turned "slope between 0.5 and 2" into simple limits on each step δ. With knots 9.95 bins apart: **$-4.98 \le \delta \le 9.95$**.

Without bounds

The fit wants $\delta = -30$.
Slope $1 - 30 / 9.95 \approx -2.0$: **time runs backwards**.

With bounds

δ is held at −4.98.
Slope 0.5: **slow, but forwards**. The other steps adjust around it.

How the solver (BVLS) does it: hold any step that wants to break its limit at the limit, solve ordinary least squares for the rest, and repeat until nothing changes.

Notebook § 10 · Code: lsq_linear(..., bounds=(lb, ub), method='bvls'). The real code goes via a Cholesky factor; same answer.

11 · ROBUST REWEIGHTING

Listen less to anchors that are far off

Some anchors are wrong: two different compounds with similar spectra. One anchor 30 bins off adds $30^2 = 900$ to the sum of squares and drags the fit.

A typical miss size that outliers can't inflate. Misses (1, 2, 3, 4, 100): the median is 3, distances from it are (2, 1, 0, 1, 97), their median is 1. Scale $\hat{\sigma} = 1.4826 \times 1 \approx 1.48$. (The standard deviation would be 39.)

Then repeat: fit → misses → $\hat{\sigma}$ → weights → fit again.

Miss	Huber weight	Tukey weight
1	1.00	0.96
2	1.00	0.84
3	0.66	0.66
4	0.50	0.45
100 (wrong)	0.02	0

Huber (3 rounds) shrinks big misses but never to zero. Tukey (4 rounds) drops gross ones entirely.

Notebook § 11 · Huber: weight 1 up to $1.345\hat{\sigma}$, then shrinking. Tukey: $(1 - (r/c)^2)^2$ up to $c = 4.685\hat{\sigma}$, else 0. Code: _robust_sigma, fit_joint.

12 · MAPS BETWEEN BATCHES

Every route through the shared clock gives the same answer

Three toy warps: $f_k(t) = t + 2$, $f_l(t) = t - 3$, $f_m(t) = 2t$. Their inverses undo them: f_l^{-1} adds 3 back, f_m^{-1} halves.

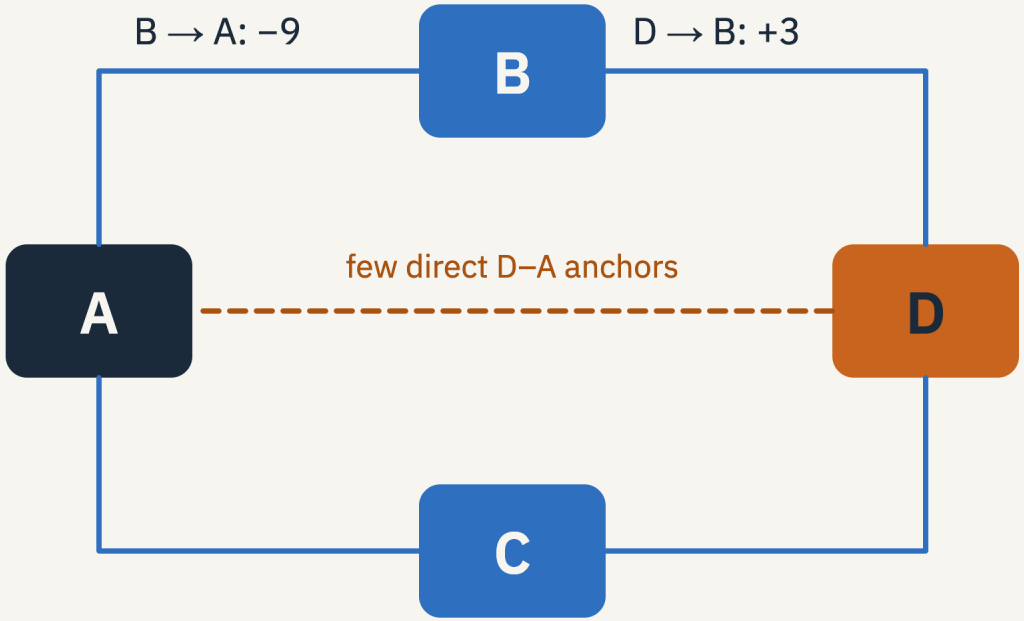
Direct k → l	$10 \xrightarrow{f_k} 12 \xrightarrow{f_l^{-1}} 15$
Via m	$10 \xrightarrow{f_k} 12 \xrightarrow{f_m^{-1}} 6 \xrightarrow{f_m} 12 \xrightarrow{f_l^{-1}} 15$

Both routes pass through 12 on the shared clock, so they **must** agree. This is called **transitivity**, and the joint model has it by construction.

Fit each pair separately and the three maps around a triangle come from three unrelated fits. On the fish data they disagree by 0.116 min on average (paper result).

Notebook § 12 · In code, f^{-1} is found by evaluating f on a fine grid and swapping the axes with np.interp. JointWarp.map(k, l, t).

A batch with few anchors is aligned through the others



If D → B is a shift of +3 and B → A is −9, then **D → A is −6**, even with no D–A anchor at all.

The joint fit uses every path at once: D → B → A and D → C → A. In the toy, batch D has only 2 samples.

The catch: with plenty of good direct anchors, a fit dedicated to one pair can follow it a little more closely.

Notebook § 13 · With all 74 direct D–A anchors, the pairwise fit is slightly better (about 0.4 vs 0.5 bins).

14 · WHERE ANCHORS COME FROM

Anchors are found by matching spectra, for every pair of samples

1. **Peaks:** the tallest points of each total ion chromatogram (at most 30).
2. **Fingerprint:** the mass spectrum around each peak, scaled to length 1.
3. **Compare** every peak in sample A with every peak in sample B by cosine similarity.
4. **Pair them up** one-to-one with the best total similarity (Hungarian algorithm).
5. **Clean up:** drop weak matches, refine to fractions of a bin, drop matches far off a robust line (RANSAC), keep only pairs in increasing order.

Refresher: cosine similarity

For length-1 vectors, the dot product is the cosine: 1 = identical spectra, 0 = nothing shared.

$(1,0,1)/\sqrt{2} \cdot (1,1,0)/\sqrt{2} = ½$

Why not just greedy?

Similarities [[0.9, 0.8], [0.85, 0.1]]. Greedy takes 0.9, then is stuck with 0.1: total 1.0. Best pairing: 0.8 + 0.85 = **1.65**.

Notebook § 14 · Peaks more than 20 min apart are never paired. Code: match_anchors, harvest_anchors. The fish data give 25,011 anchors.

15 · CHOOSING λ

Hide whole stretches of time, then try to predict them

Cross-validation: hide part of the anchors, fit on the rest, score how well the fit predicts the hidden ones. Try each λ; keep the best score. The truth and the evaluation metric are never used.

Why hide time, not samples? Hidden samples leave anchors at the same compounds' times as the training anchors, so a wiggly drift that passes through every compound still looks perfect.

Retention time cut into blocks of 12 bins; block b goes to fold b mod 5:



Bins 0–11, 12–23, …, 108–119: the first ten blocks, labelled with their fold.

Example: t = 30 is in block 2 → fold 2; t = 70 is in block 5 → fold 0. The anchor (30, 70) is hidden in both folds 2 and 0. On the fish data this chose **λ = 10**.

16 · BOOTSTRAP UNCERTAINTY

Refit on resampled data to see how much the warps move

1. Resample

Within each batch, draw samples with replacement. Counts (2, 0, 1): sample 0 twice, sample 1 never, sample 2 once.

2. Reweight, refit

An anchor's weight = the product of its two samples' counts: 2 × 1 = 2. Refit, and repeat 200 times.

3. Read off a band

The 5th–95th percentile of the refitted maps at each time: a 90% band. Add anchor scatter for a predictive interval.

Honest result: the intervals are too narrow. On the fish data the 90% predictive intervals contain 81% of held-out anchors (paper result).

Why: a bootstrap sees how much a fit *varies*, not a *bias* every refit shares. The smoothness penalty pulls every refit towards a straighter drift the same way, so the band sits in slightly the wrong place.

Notebook § 16 · Predictive half-width = $\sqrt{(\text{half-band}^2 + (1.645\,\sigma)^2)}$, σ the pair's robust scale. Calibration (e.g. conformal) is the suggested fix.

THE WHOLE RECIPE

One least-squares problem, with four additions

$$\text{minimise } \Sigma\, w_j\, r_j^{\,2} + \lambda' \cdot \text{bending} + \gamma \cdot \text{gauge}$$

keeping every slope between 0.5 and 2

r_j
one miss per anchor, linear in θ
§ 3–7

w_j
robust weights, updated each round
§ 11

bending
squared second differences; λ by blocked CV
§ 9, 15

gauge
pins where the shared clock sits
§ 8

slope limits
box bounds on each step δ
§ 4, 10

Then every map between batches is $\mathbf{g(k \to l) = f_l^{-1} \circ f_k}$ (§ 12), consistent by construction, with a bootstrap band (§ 16).

Section numbers refer to tutorial/mba_from_scratch.ipynb. § 18 checks the from-scratch version against src/multibatch.py.